SentinelOps: An AI-Powered Autonomous DevOps Platform for End-to-End Software Delivery, Intelligent Infrastructure Operations, and Self-Healing Automation

R. Nivedha
Assistant Professor
Dept. of Computer Science and Engineering, Sathyabama Institute of Science and Technology, Chennai, India
nivedha.cse@gmail.com

P N Praveen
Student
Dept. of Computer Science and Engineering, Sathyabama Institute of Science and Technology, Chennai, India
ppraveen2150@gmail.com

Subin VM
Student
Dept. of Computer Science and Engineering, Sathyabama Institute of Science and Technology, Chennai, India
subinvm@gmail.com

S. Nivetha
Assistant Professor
Dept. of Computer Science and Engineering, Sathyabama Institute of Science and Technology, Chennai, India
nivethacse555@sathyabama.ac.in

Hemavathi
Assistant Professor
Dept. of Computer Science and Engineering, Sathyabama Institute of Science and Technology, Chennai, India
hemavathi.s.cse@sathyabama.ac.in

A. Annie Micheal
Assistant Professor
Dept. of Computer Science and Engineering, Sathyabama Institute of Science and Technology, Chennai, India
annymick@hmail.com

***Abstract*—Moving a project from source code to a working delivery pipeline is slow for student and small-team developers, because the facts needed to package, test, deploy and monitor an application are spread across manifests, scripts and folder layouts. This paper presents SentinelOps, an AI-powered desktop platform that automates that path for an unfamiliar Git repository. The platform clones the repository, infers its languages, frameworks, service boundaries and start commands from observable evidence, and returns the result to the user for correction. After the application has been validated locally, SentinelOps generates Docker and Compose files, GitHub Actions, GitLab CI or Jenkins pipelines, Kubernetes manifests with a Horizontal Pod Autoscaler, and Prometheus and Grafana configuration. A language-model recommender proposes stacks, delivery profiles and error explanations, and an approval-gated self-healing loop links health signals to runbook actions. Autonomy is applied to discovery, generation, validation and diagnosis, while any step that changes a running environment waits for user consent. We describe the architecture, the project state model, the module boundaries and a plan for evaluating detection accuracy and operational transparency. No benchmark results are claimed at this stage.**

***Index Terms*—DevOps automation, AIOps, self-healing systems, repository analysis, CI/CD, containerization, Kubernetes, observability.**

I. Introduction

Interest in applying artificial intelligence to software delivery has grown quickly [1]. Yet a small team can build an application, run it on a laptop and still stall when it is time to package it, pick a place to run it, add continuous integration and watch the running service. The information those steps need is scattered across package files, scripts, environment files and the repository layout. A developer who is new to a project can easily miss, for instance, that a React client and its API must be started as two separate processes. Continuous delivery practice deals with part of this gap by treating build, test and deployment as one delivery process rather than a manual step added at the end [2].

SentinelOps is our response to that gap. It is an AI-powered desktop platform that automates the route from a Git repository to a reviewed delivery plan and then keeps watching the result. The word *autonomous* in the title needs a precise reading. SentinelOps works on its own during discovery, generation, validation and diagnosis, but it never changes a running environment without the approval of the user. Deterministic repository evidence comes first. An editable model of the detected stack follows, and a language-model recommender and a self-healing loop sit on top of that model and stay within its limits. The ordering is deliberate. A tool that automates delivery should be able to explain why it inferred a technology or a command, and the project owner should be able to correct that inference before any delivery file is written or any container is started.

This paper makes four contributions. First, it describes an evidence-first pipeline that turns a Git repository into an editable project model. Second, it separates local validation from asset generation, so a broken application is caught before any deployment file exists. Third, it defines delivery profiles, observability assets and an approval-gated self-healing loop that keep every generated artifact and every corrective action open to review. Fourth, it gives a concrete plan for evaluating repository analysis and operational transparency. The remaining sections cover related work, design goals, architecture, implementation, the user workflow, delivery profiles, observability and self-healing, the evaluation plan, limitations, a demonstration scenario and conclusions.

II. Related Work and Motivation

A. Continuous Delivery and Containers

Continuous delivery is usually defined as the ability to keep software in a releasable state through automated build, test and deployment [2]. Small projects, however, often begin with nothing more than a source repository and a local start command. The distance from there to a repeatable delivery process is larger than it looks. It covers dependency installation, environment configuration, packaging, test

selection, image publishing, the choice of a deployment target and operational visibility, and each step depends on details that may be only implicit in the code.

Containers take care of one part of the problem. They bundle an application with its runtime dependencies, and the Docker model has been widely adopted to narrow the differences between developer machines and deployment hosts [3]. A Dockerfile, though, cannot be written safely from a file extension alone. A front-end project may need a build stage and a static server, while an API may need a process manager, a database connection and a health endpoint. For that reason SentinelOps treats every container template as a draft for review.

B. Observability and Scaling

Observability supplies the evidence needed once an application is running. Prometheus stores metrics as labeled time series and pulls them from monitored targets [4]. In a Kubernetes cluster, the Horizontal Pod Autoscaler (HPA) changes the replica count in response to observed resource or custom metrics [5], and Grafana organizes queries into dashboards that help a user inspect the state of a service [6]. These tools help only when their assumptions are visible. A dashboard with no reachable target, or an autoscaler fed by unreliable metrics, does not make a service any safer.

C. AI-Driven DevOps and Project Scope

Research on AI-driven DevOps concentrates on decision support, anomaly detection and automated remediation [1]. SentinelOps touches all three, each at a different level of autonomy. A recommendation layer proposes a stack or a deployment profile, a health-and-threshold loop detects abnormal conditions, and a remediation step proposes a runbook action. The repository model, the generated files, the commands and the warnings all stay open to inspection, and actions that change state wait for approval. This suits project work, where students must explain their design choices during demonstrations, reviews and submissions.

The question addressed here is a practical one: how can a desktop application lead an unknown Git repository along a transparent path from inspection to locally validated delivery assets, and then to supervised operations? The answer is neither a single model nor a single deployment target. It is a chain of evidence-based modules with clear handoffs and a point at which the user can correct anything the tool is unsure about.

The prototype rests on established delivery and platform practice. Docker distinguishes an image from a running container, and Compose describes multi-container applications [7]. Git supplies the revision and remote metadata needed for traceable intake [8]. GitHub Actions, GitLab CI and Jenkins each let a team describe a pipeline inside the repository itself [9]–[11]. Kubernetes Deployments and Services provide the workload and network abstractions behind the Kubernetes profile [12], [13], and OpenTelemetry defines a vendor-neutral way for an application to expose metrics and traces [14].

D. Security and Provenance

Security and provenance are design constraints here, not claims the tool makes on its own. NIST guidance on application containers describes the risks specific to container platforms [15], supply-chain guidance recommends recording and reviewing build inputs [16]–[18], and the Secure Software Development Framework sets out secure development practices [19]. Earlier studies of containerization and of security in DevOps informed the same decisions [20], [21]. Electron and Node.js provide the desktop and runtime foundations of the implementation [22], [23], and work on infrastructure as code and cloud-native security guided the structure of the generated templates [24], [25]. Even so, every artifact remains editable and needs review by the project owner.

III. Design Goals and Scope

SentinelOps was built around five design goals. The first is to recognize common project shapes without assuming that a repository holds a single service. The second is to keep local verification apart from deployment generation, so that a broken application shows up early. The third is to produce ordinary, editable artifacts instead of hiding configuration behind a graphical interface. The fourth is to keep changes to the host machine small: Docker Desktop is checked as an external prerequisite on Windows and is never installed by the application. The fifth is to make operational choices visible through a deployment profile, health checks, scaling settings and an observability plan.

The prototype targets Node.js, React, Next.js, Vue, Angular, Python, Flask, FastAPI, Django, Go and Rust projects. Its detector examines package manifests, lock files, framework dependencies, source layout, scripts, ports and existing container configuration. The result is a proposed model, not a guarantee. Before proceeding, the user can edit the detected language, framework, service type, runtime commands, ports and CI provider.

Because the title promises autonomy, the scope of that autonomy is stated in Table I. Four levels are defined. Levels L0 and L1 run without further prompting, since they only read files, produce text or run commands the user has already reviewed. Level L2 covers actions on the local machine and needs approval for each action. Level L3 covers remote and production changes; it lies outside the prototype and is listed only to mark where the boundary sits.

TABLE I
Levels of Autonomy in SentinelOps

Level	Scope	Examples	Approval
L0	Observe and propose	Repository inspection, stack detection, file generation, recommendations	Not needed
L1	Validate locally	Dependency install, service start, health probe	User reviews commands first
L2	Act locally	Restart or stop a process, return to an earlier generated configuration	Required for each action
L3	Act remotely	Image push, cluster deployment, cluster scaling	Out of scope

IV. System Architecture

Fig. 1 shows the layered design. A desktop interface sits on top, an orchestration layer holds the project state and the approval gate, a set of single-purpose modules does the work, and the external tools it relies on sit at the bottom. Repository intake produces a local workspace together with Git metadata, and the analysis module builds an evidence-backed project model from them. Local verification follows. It installs the declared dependencies, starts the service with the selected command, probes a health endpoint when one exists and records the URLs a user can open in a browser. The asset generator prepares container, pipeline, deployment and monitoring files only after that gate has been passed.

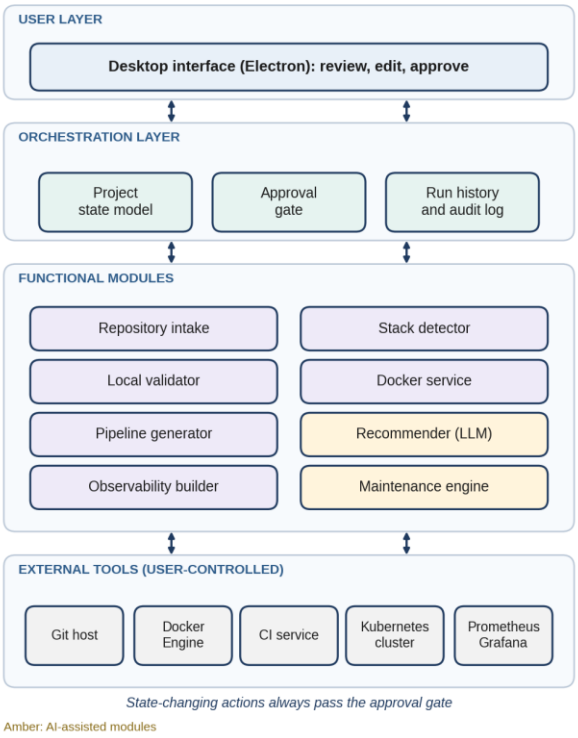


Fig. 1. Layered architecture of SentinelOps. Amber modules use a language model; all others are deterministic.

The orchestration layer is not meant to replace a production platform. Its role is to coordinate the choices made by the user. A local profile, for example, can favor a native process for early testing, whereas a container profile generates Compose assets for a reproducible local environment. The Kubernetes profile adds a Deployment, a Service and a HorizontalPodAutoscaler manifest. Kubernetes describes the HPA as a controller that adjusts workload replicas according to observed metrics [5], so SentinelOps presents scaling thresholds as a policy to be reviewed, never as an unexplained automatic action.

A. Project State Model

Every project moves through the sequence of states drawn in Fig. 2. The initial state holds a repository address and a project name. Once the repository is cloned, the model gains metadata such as the remote URL, current branch, commit identifier, file inventory and detected components. The review state records the commands, ports and target profile chosen by the user. The validated state adds command output, a health result and local URLs, and the generated state stores the locations of the Docker, pipeline, deployment and monitoring files. Keeping these transitions gives the user a short history of how a delivery decision was reached.

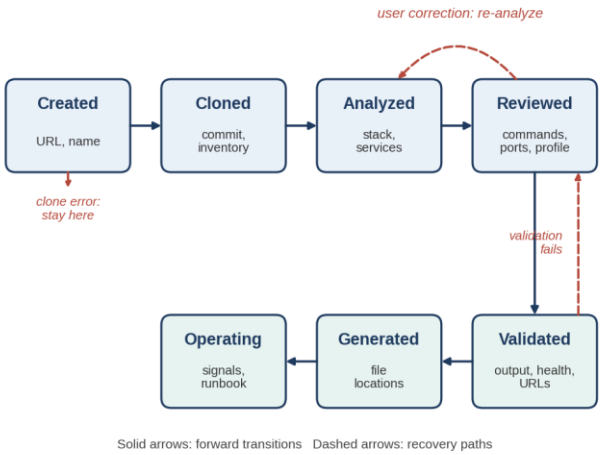


Fig. 2. Project state model with forward transitions and recovery paths.

B. State-Oriented Interface

The desktop interface is organized around these states instead of forcing the user through a hidden linear wizard. A repository can be analyzed again after a correction, and a generated plan can be modified without cloning the project a second time. This matters when a repository uses a nonstandard script name, a custom port or a monorepo layout. The user should be able to repair the model at the point where an assumption is wrong, rather than editing a downstream file and losing the link to the original evidence.

C. Handling of Private Information

Private information is kept apart from generated configuration. Source URLs and local paths may be stored in the project record, but registry credentials, deployment secrets and cluster tokens are never guessed by the detector. If a later integration needs such values, it should request them through the interface and pass them on through secure, host-specific storage. This separation lowers the chance that generated examples encourage users to commit secrets to a repository.

V. Implementation Details

A. Repository Intake and Detection

The application is built around a small set of modules, summarized in Table II. The Git module clones a repository into a project-specific directory and, after confirmation, can delete that cloned workspace. The detector reads repository files and draws its findings only from observable evidence. When it finds independent package manifests or framework signatures, it reports the client and server directories separately, which lets the interface show two start commands and two local addresses for a project that needs them.

TABLE II
SENTINELOPS MODULES AND THEIR OUTPUTS

Module	Output or decision
Repository intake	Cloned workspace, commit metadata and project inventory
Stack analysis	Editable language, framework, package and front-end/back-end findings
Verification	Install and start commands, health result and local access URLs
Delivery assets	Dockerfile, Compose file, CI workflow, Jenkinsfile or GitLab pipeline
Operations	Deployment profile, HPA manifest, Prometheus configuration, dashboard JSON and runbook
Recommendation	Stack, profile and error explanations with evidence and confidence notes
Maintenance	Runbook suggestions and approval requests for corrective actions

Analysis begins with the files that state project intent explicitly (Fig. 3). For JavaScript and TypeScript projects, the detector reads `package.json`, lock files, declared scripts and framework dependencies. For Python projects it examines requirements files, `pyproject` metadata, common entry files and framework imports. Comparable rules recognize Go modules, Rust manifests and existing container files. File evidence is then combined with directory evidence, so a client folder with its own manifest beside a server folder with a different one is reported as a split application, not as a single ambiguous project.

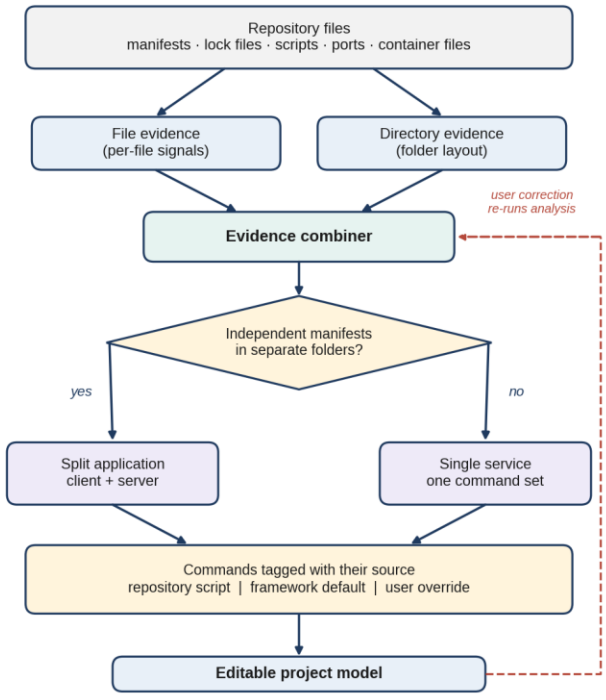


Fig. 3. Evidence-based detection flow, from repository files to an editable project model.

Recommended commands are derived from the manifests that were inspected. A declared development script, for instance, is stronger evidence than a generic framework default, and SentinelOps records the source of each command in the project model. The interface can therefore tell a direct repository script from a fallback suggestion. If the user changes a command by hand, the revised command becomes the value used for local validation and later asset generation, while the original detection remains visible for comparison.

B. Local Validation

The validation module resolves the selected install command, starts the application with the chosen runtime command and captures the output for the user. It is deliberately separate from Docker execution. The module first confirms that the selected runtime is available, runs the dependency installation in the cloned workspace and then starts the service as a managed process. Output is streamed to the desktop interface and stored with the validation record. Where a web address can be inferred, SentinelOps probes the endpoint and shows a clickable local URL. For a project with separate client and server services, the result lists a process state and a URL for each, rather than a single generic success message.

The module reports the process state and the local URL, and it does not declare the application ready merely because a command was launched. A successful probe is useful evidence, but it covers only the chosen endpoint in the local environment. Keeping that distinction clear prevents generated deployment assets from being mistaken for a verified production release.

C. Containers and the Docker Service

For containerized execution, SentinelOps derives a Dockerfile template from the confirmed runtime. Docker packages an application together with the libraries and configuration it needs, which reduces differences between development and deployment environments [3]. The generated file remains editable because build steps, secrets, multi-stage builds and operating-system packages differ from project to project.

Before offering any image-related action, the Docker service runs a host status check. On Windows, SentinelOps tells the user that Docker Desktop must be installed and running; it does not try to install the software or modify the host. When Docker is available, the project can pull a chosen base image, build an application image and run a container with the proposed port mapping. The Dockerfile and Compose file are displayed as ordinary text, so the user can change base images, build arguments, environment variables, volumes or service names before anything runs.

D. Pipeline Generation

The pipeline module chooses among GitHub Actions, GitLab CI and Jenkins according to the repository context and the preference of the user. It generates a visible sequence of checkout, dependency installation, build, test, image build and an optional publish step. Credentials, registry URLs and production deployment commands are deliberately not inferred, because they require values that only the user or the organization can supply.

E. Recommendation Layer

A freely available language model serves as a secondary recommender for a stack, a delivery path or an explanation of an error. It works inside the limits of the repository model. It must not invent a framework that the detector never observed, run shell commands without approval, or overwrite generated files without keeping the previous version. In this design the model output is a proposal accompanied by evidence and confidence notes, while deterministic checks stay responsible for file discovery, command execution and validation status. The split lets the platform use AI for judgment calls without allowing a plausible but wrong answer to reach the machine of the user unchecked.

F. Visualization and Error Handling

The project also includes a visualization view of the chosen technology stack and the planned delivery path. It maps repository findings to the selected CI provider, container base image, deployment profile, monitoring assets and response policy. The purpose is operational clarity. A user can see, for example, that a FastAPI service was paired with Python, Docker, a GitHub Actions workflow, a Kubernetes profile, Prometheus scraping and a CPU-based HPA before any remote deployment command is considered.

Error handling is built around recovery instead of silent retries. A clone error leaves the project in the intake state and shows the Git message. A detection error keeps the repository record and asks the user to fill in the fields manually. When an install or a local start fails, the terminal output is stored and the interface does not present deployment generation as a verified path. A Docker error reports either the missing host prerequisite or the result returned by the engine. These rules keep each failure trace next to the action that caused it.

VI. User Workflow and Controlled Execution

A session starts when the user creates a project and supplies a Git repository address. The application records a readable project name and selects a local directory for the clone. If the repository already exists in an earlier project workspace, the user can inspect its metadata or remove the cloned copy through a dedicated delete action. Deletion affects only the copy managed by SentinelOps and must never remove the original repository of the user or any remote repository. The constraint is small, but it matters because people often try several repositories while learning a new tool.

After cloning, the analysis screen shows the detected stack as editable fields rather than a black-box result. The fields cover the primary language, framework, package manager, build tool, service type, detected ports, container support and CI configuration. When a repository contains both a front end and a back end, each component gets its own entry. A user can confirm a React client on one port and a Python API on another, and those selections carry over when Docker or pipeline assets are created.

Local validation comes next. The user reviews the install and start commands before running them, and SentinelOps then displays the terminal output, process status and local addresses. This is especially helpful for projects whose front end and back end must be launched separately. Instead of asking the user to work out an address from the logs, the application shows explicit links, such as a client URL and an API URL, whenever they are known. If a process stops unexpectedly, its exit result becomes part of the validation record.

When validation succeeds, or when the user chooses to continue despite a known limitation, the planning screen recommends a delivery profile. The recommendation considers the selected stack, whether the service is split, whether a Dockerfile already exists and the target stated by the user, and the screen explains its reasoning in plain language. It may suggest native local execution for an early single-service prototype, Compose for a service with a local dependency, or Kubernetes only after a container image and a cluster target have been confirmed.

The execution screen keeps generated files clearly apart from commands that have actually run. Creating a Dockerfile does not mean that an image exists. Generating a Jenkinsfile does not mean that a pipeline has been configured on a Jenkins server, and preparing a Prometheus configuration does not mean that metrics are being collected. Each stage therefore carries one of the six statuses defined in Table III. Showing this distinction stops an attractive visualization from overstating the state of the system.

TABLE III
Execution Status Model

Status	Meaning
Generated	The file exists in the workspace; nothing has been executed.
Configured	The file has been applied to a tool or service, for example imported into a CI server.
Started	A process, container or monitoring stack has been launched.
Validated	A probe or check has confirmed the expected behavior in the local environment.
Failed	A step ran and returned an error; the output is stored with the record.
Not applicable	The stage does not apply to the chosen profile.

The final project view works as a compact record for demonstrations. It can show the repository source, selected stack, local test result, container choice, CI/CD template, deployment profile, observability configuration, scaling policy and recommended maintenance actions. The same record gives a reviewer enough context to understand why the application chose its assets, and it serves as a handoff note when another developer must continue the setup without repeating the initial investigation.

VII. Delivery Profiles and CI/CD

A. Profiles and Their Selection

SentinelOps presents deployment as a choice among profiles, not as a single action that suits every project. The native local profile is the simplest: it starts the selected command directly on the workstation and is convenient for debugging a freshly cloned repository. The Docker Compose profile packages the application and its supporting services

into a repeatable local bundle. The Kubernetes profile is intended for users who already have an approved cluster and understand what a remote deployment involves. Table IV summarizes the differences, and Fig. 4 shows how the recommendation is reached.

TABLE IV
DEPLOYMENT PROFILES OFFERED BY SENTINELOPS

Profile	Appropriate use	Generated guidance
Native local	Early debugging on one workstation	Commands, URLs, health status and stop action
Docker Compose	Repeatable local service bundle	Dockerfile, Compose file, image and port mapping
Kubernetes	Cluster deployment with approval	Deployment, Service, HPA and capacity policy

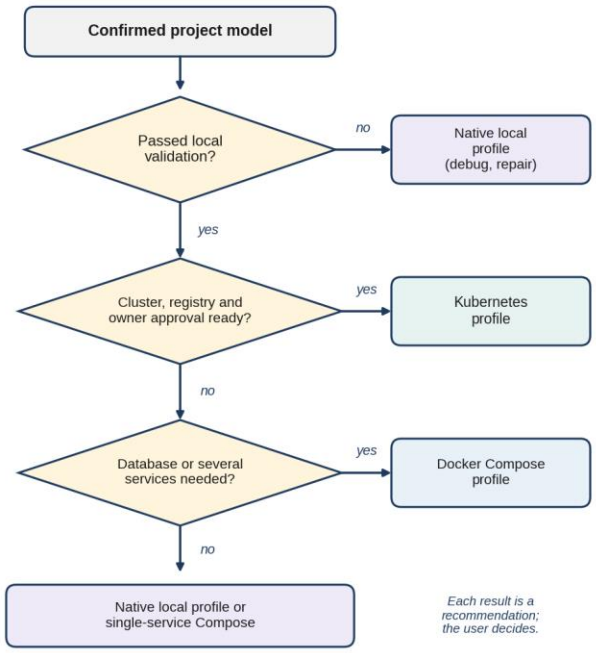


Fig. 4. Decision flow used to recommend a delivery profile.

The recommended profile depends on the confirmed project model. A small static front end can often be tested locally and then built into a compact web-serving image. A service that needs a database may be better served by Compose during development, since the application and its dependency can be described together. A split front end and back end needs separate service entries, port mappings and health checks. Kubernetes becomes a reasonable recommendation only when a cluster, an image registry and operational ownership are already in place, and SentinelOps lists these prerequisites in the profile description instead of presenting Kubernetes as the natural next step. Fig. 5 lists which artifacts each profile produces.

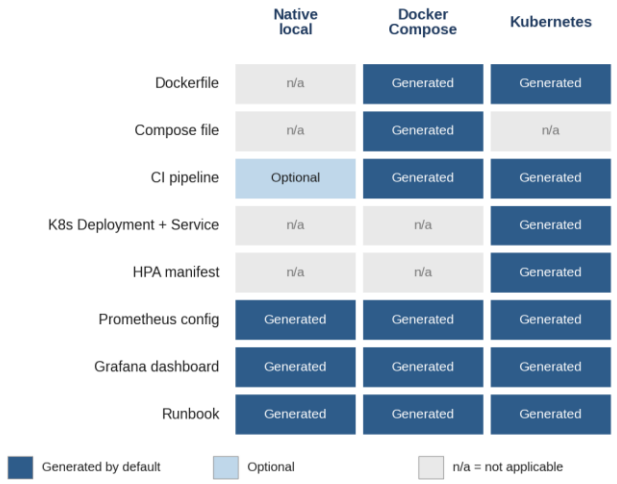


Fig. 5. Artifacts produced by each delivery profile (design coverage, not measured results).

B. Pipeline Templates

The CI/CD templates follow the same principle. A GitHub repository can start with GitHub Actions, a GitLab repository with GitLab CI and a Jenkins environment with a Jenkinsfile. Each template begins with checkout and dependency installation and then runs the project build and the selected tests. Container build and push steps stay optional because an image registry needs a destination and credentials supplied by the user. A deployment stage can also be generated, but it remains disabled until the user provides the target environment and the approval process.

The generated pipelines are written so that a developer who did not build SentinelOps can still read them. Steps carry clear names and use standard repository scripts wherever possible. A pipeline should fail when a build or test fails, and its report should point back to the command output. The application does not promise to repair every failure on its own. It can identify the failed stage, collect the message, explain the usual causes and suggest what to check next, which is safer than retrying a build indefinitely or changing source code without review.

C. Image Tagging

Image tagging is handled as a release decision. A generated configuration can use a local tag for testing and can reserve immutable tags derived from a commit identifier for an approved registry workflow. A later production integration should add provenance, vulnerability scanning and access control before an image is promoted. SentinelOps can prepare the surrounding files, but registry permissions and the policy that decides whether an image may reach an environment remain the responsibility of the project owner.

VIII. OBSERVABILITY, AUTOSCALING AND SELF-HEALING

A. Metrics and Dashboards

SentinelOps creates a Prometheus scrape configuration, a starter Grafana dashboard definition and an optional Compose stack for local observation, building on the roles of Prometheus and Grafana described in [4] and [6]. The

interface summarizes the generated endpoints and suggests signals worth watching, such as service availability, CPU use, memory use, request errors and restart count. It does not claim to show live metrics until the user has started the monitoring stack and the application exposes a compatible metrics endpoint.

The configuration starts from a short, explicit target list. The user chooses the application endpoint and the scrape interval, and SentinelOps writes a Prometheus configuration that can be reviewed before it is started. A starter dashboard groups panels for availability, CPU, memory, process restarts and request errors whenever the chosen runtime exposes those metrics. The dashboard is a starting point, not proof that every application uses the same metric names. A Python API, a Node.js process and a static client each expose different signals, and some may need an instrumentation change, for example through OpenTelemetry [14], before the panels show any data.

B. Autoscaling

Autoscaling is offered only for the Kubernetes profile. SentinelOps generates an autoscaling v2 resource with reviewed minimum and maximum replica values and a target CPU utilization. It also adds resource requests to the generated workload, since CPU-based scaling depends on them [5]. The user can adjust the capacity policy in the interface and see the resulting manifest. The system also points out that heavy traffic alone does not guarantee a scale-out event, because the configured metric, the target, workload readiness and cluster capacity all influence the outcome.

C. Approval-Gated Self-Healing

The maintenance model is recommendation-led. When a health probe fails or an observed threshold is crossed, the interface can propose a restart, a rollback, a scaling change, a configuration review or a ticket, and a runbook explains the suggested action together with the evidence that triggered it. Automatic source-code repair and unapproved production changes are outside the scope of the prototype. This boundary suits a desktop tool used by learners and small teams, where a wrong assumption could otherwise alter a local project or a remote environment without warning.

The self-healing loop is drawn in Fig. 6. A health or metric signal flags an abnormal condition (step 1). The system then gathers context such as the selected profile, recent command output, the configured endpoint and the last known status (step 2), and matches the event to a runbook suggestion (step 3). Before any action that changes a running environment, such as a local restart, a rollback to an earlier configuration, a scaling adjustment or a notification, it asks the user for approval (step 4). An approved action is applied (step 5) and every outcome, including a declined request, is recorded and fed back into monitoring (step 6). The loop keeps each suggested response explainable and reversible. In the current prototype the trigger is a health probe or a configured threshold. A learned anomaly detector is a natural extension, and Section X returns to it.

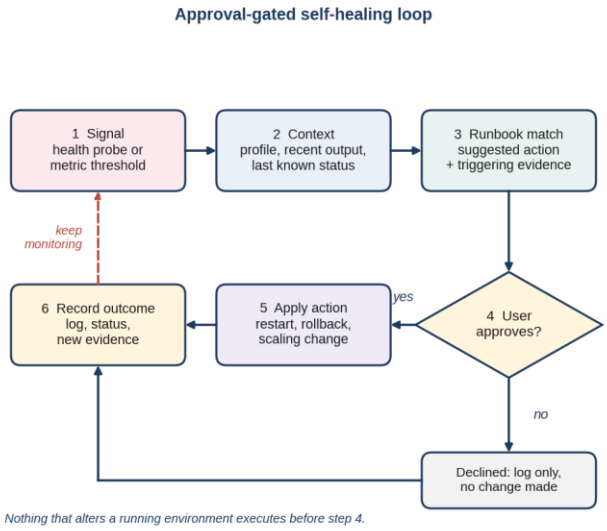


Fig. 6. Approval-gated self-healing loop.

D. Notifications and Visual Summary

Notifications are treated as an integration boundary, not as a service that can be assumed to exist. The project model can reserve hooks for email, Slack, Microsoft Teams, Jira or PagerDuty, but a real connection needs credentials, endpoint configuration and an escalation policy specific to the organization. In the current prototype the user receives the event record and the recommended action. This keeps the paper consistent with what SentinelOps can safely generate and avoids suggesting that an alert was delivered when no integration has been configured.

The desktop visualization follows the same maintenance model. A generated plan can show the container image, CI/CD stages, deployment profile, metric sources, autoscaling range and latest health result as connected modules. It does not replace Grafana. Its job is to explain how the generated assets relate to one another before the user opens a monitoring dashboard, which helps in project demonstrations because viewers can follow the path from repository analysis to an operational policy without needing live cloud credentials.

IX. Evaluation Plan

A. Detection Accuracy

The prototype is to be evaluated on a curated set of public repositories covering single-service Node.js and Python applications, front-end applications and repositories with separate client and server folders (Table V). For each repository, an evaluator records the expected language, framework, package manager, start command, ports and service boundaries before running SentinelOps. Each detected field is then scored against that record. A field that matches counts as a true positive (TP), a field that is detected but wrong or unsupported counts as a false positive (FP), and an expected field that is missing counts as a false negative (FN). Detection precision, recall and their harmonic mean follow from these counts:

$$P = TP / (TP + FP) \quad (1)$$

$$R = TP / (TP + FN) \quad (2)$$

$$F_1 = 2PR / (P + R) \quad (3)$$

Two further measures complete the picture: the rate of correct local launch instructions, and the manual correction rate

$$C = N_{corr} / N_{gen} \quad (4)$$

where N_{corr} is the number of generated artifacts that need a manual edit and N_{gen} is the number of artifacts generated.

TABLE V
PROPOSED REPOSITORY EVALUATION CASES

Project shape	Expected finding	Evidence reviewed
Single API	Runtime and port	Manifest, scripts and source entry point
Web client	Framework and build command	Dependencies, lock file and public assets
Client plus server	Two services and URLs	Independent manifests and folders
Container-ready repository	Existing files preserved	Docker and Compose configuration

B. Operational Transparency

A second evaluation examines operational transparency. Participants should be able to identify the selected deployment profile, the command used for local validation, the container image tag, the metrics endpoint and the condition that would trigger scaling. A short user study could compare this workflow with a manual checklist. The aim is to find missing evidence and confusing defaults, not to make unsupported claims about deployment speed or reliability.

C. Manual Correction and Failure Tests

The evaluation should also score how well manual correction works. An evaluator deliberately changes a command, a port or a framework field and then checks whether the regenerated Dockerfile, pipeline and deployment artifacts reflect the new value. A good result is not just a high automatic detection rate. It is a tool that stays coherent after a user corrects the detector, which matters because repositories often contain legacy scripts or examples that should not be chosen as a production entry point.

Failure-oriented tests are equally important. The test set should include an invalid Git URL, a private repository without access, a dependency-install failure, a missing runtime, a server that starts but never answers the health probe, a missing Docker engine and a missing metrics endpoint. For each case the evaluator records whether SentinelOps preserves the project state, exposes the source of the error, avoids showing a misleading ready status and offers a useful next action. The same suite should include a declined approval request, to confirm that the self-healing loop leaves the environment untouched. These tests measure the reliability of the workflow itself, not the performance of any particular application.

D. Reporting

No numeric results are reported in this paper, because the full benchmark set and the participant study have not yet been completed. The prototype demonstrates the architecture and the generated artifacts, and the proposed study defines how accuracy and usability will be measured later. Reported results should include the repository sample, exact software versions, manual overrides and failures, so that another student team can repeat the comparison.

X. LIMITATIONS AND FUTURE WORK

A. Limitations and Security Requirements

Repository inspection cannot reliably discover every runtime requirement. Private registries, undocumented environment variables, external databases and shell scripts may all need manual input. Stack selection also depends on the quality of the underlying evidence and on the optional language-model recommendation, which is why SentinelOps records both its own detection and the final selections of the user. The AI component of the current prototype is limited to recommendation and explanation, and failure detection relies on health probes and configured thresholds rather than a trained model. Future work could add signed artifact provenance, secret scanning, configurable policy checks, richer service discovery and a controlled approval workflow for remote deployment.

More security work is needed before SentinelOps is used outside a controlled development setting. Generated containers should run with least privilege, use small and maintained base images and avoid embedding secrets. CI/CD credentials belong in the secret store of the provider, not in generated YAML files. A production deployment path should include access review, image scanning, signed releases and a rollback plan. These controls are stated here as requirements, not as capabilities the tool already has.

B. Scope of Automation and Future Directions

The current scope excludes autonomous code changes. A language model can help interpret logs or describe a likely configuration problem, but it may propose a fix that sounds plausible and is wrong. SentinelOps therefore limits automated behavior to inspection, generation, local process control, health observation and recommendations that a user can review. Future versions could add a sandboxed repair branch, a test-only validation gate and a pull-request workflow for proposed changes. That approach would keep human review in place while making recovery experiments more systematic. A learned anomaly detector trained on the health and metric history of a project could also replace fixed thresholds as the trigger for the self-healing loop, provided its alerts stay explainable.

Richer architecture discovery is another direction worth pursuing. A repository may contain infrastructure modules, database migrations, message queues, mobile clients or several deployable services. Representing these components explicitly in the project model would improve deployment recommendations, and the same model could support cost-aware choices, environment-specific configuration and dependency graphs. Any such extension should preserve the present principle that generated conclusions can be traced to files, user selections or stated operational policies.

XI. Demonstration Scenario

A typical session starts with a repository that contains a React client and a Python API. The user creates a project, pastes the repository URL and starts the clone. When the clone finishes, the Git module shows the remote address and the local workspace path. The detector then finds two independent package descriptions and reports a client component and a server component. It proposes a JavaScript front-end framework, a Python API framework, separate dependency commands and two ports, and the user can revise any of these before they become part of the project plan.

Next, the user runs local validation. SentinelOps installs the front-end and back-end dependencies in their own folders, starts each service and streams the output to the workspace view. The front-end URL and the API URL are reported separately. If the front end loads while an API route fails, the record shows that difference instead of treating the project as fully healthy. A developer can use the captured output to change an environment variable, a port or a start command and then repeat the test. The scenario shows why separate service discovery is worthwhile for full-stack repositories.

After a successful local check, the planner can recommend Docker Compose as the first deployment profile. It prepares one service definition for the client and one for the API, each with its chosen port and a shared network. The user reviews the proposed Dockerfiles, selects a supported base image and starts image pull or build actions only if Docker is ready on the host. On Windows, the Docker status view makes this prerequisite visible. There is no hidden installation path, and the interface does not claim that a container was built before the engine returns a result.

For source delivery, the user selects the CI provider that matches the repository host. A GitHub repository can receive a GitHub Actions workflow with checkout, dependency installation, build, test and optional image-build stages. A GitLab repository can receive a comparable GitLab CI definition, and a Jenkins environment can receive a Jenkinsfile that its administrator may import into an existing server. The template stays a generated file in the project workspace. Connecting it to a remote service is a later and deliberate step that needs credentials and repository permissions.

If the user later chooses the Kubernetes profile, SentinelOps converts the selected service information into editable workload manifests. The plan can include a Deployment, a Service, resource requests and an HPA with an explicit replica range. The interface explains that this profile needs a cluster, an image registry and access approval, and it makes clear that the HPA follows the selected resource policy. It does not suggest that traffic will automatically trigger a cloud deployment, or that the desktop application can scale a cluster it cannot reach.

The observability step adds a local Prometheus configuration and a starter Grafana dashboard definition. Once the user starts the monitoring services and exposes application metrics, the plan can connect the application target to the dashboard panels. A failed scrape appears as a missing dependency or a configuration problem and never as a successful observation. The maintenance view can then link a failed health check or a high-utilization signal to a runbook entry, so the user sees the evidence, the suggested action and whether approval is required before any corrective operation.

The scenario illustrates what automation means in SentinelOps. The application takes over the repetitive discovery and configuration work but does not hide decisions that carry operational risk. A student can demonstrate the entire flow, from repository to generated artifacts, on a single machine. A small team can use the same flow to build a consistent starting point before involving a registry, a CI server, a cluster administrator or a production monitoring environment.

XII. Reproducibility and Maintainability

Text-based outputs are preferred because they are easy to inspect, compare and version alongside the source code. Dockerfiles, Compose files, CI pipelines, Kubernetes manifests, Prometheus configurations, Grafana dashboard JSON and runbooks should all be stored in a predictable project folder. SentinelOps records the repository revision used for analysis and the settings selected for generation. If a user regenerates an asset after changing the stack model, the application can compare the new configuration with the earlier one and show which assumptions changed.

A maintainable tool also needs predictable cleanup. SentinelOps gives each cloned repository its own project workspace and a specific option for deleting that copy. It should stop any managed local processes before removing the workspace and should name the target directory clearly before asking for confirmation. Generated assets can be kept after the clone is removed if the user wishes to retain a design record, so disk space can be recovered without losing the history of the chosen deployment plan.

For academic use, repeatability depends on documenting the host assumptions. A project demonstration should state the operating system, runtime versions, Docker availability, repository commit, selected commands and chosen profile. Screenshots or exported project summaries can capture the detected stack and the generated flow. With these details another evaluator can separate a product limitation from a missing host prerequisite, which also makes the evaluation proposed in Section IX more credible.

XIII. Conclusion

SentinelOps offers a practical path from a Git repository to a reviewed DevOps plan and a supervised operating loop. The central idea is simple: inspect the project, confirm that it runs locally and only then generate assets for containers, CI/CD, deployment and monitoring. The autonomy of the platform is real but bounded. It discovers, generates, validates and diagnoses on its own, and it leaves every decision that changes a running environment to the user. These properties make it suitable for learning, for project demonstrations and for small teams that need a repeatable starting point for delivery automation. The planned evaluation on public

repositories will show how well the detector and the generated assets hold up in practice.

REFERENCES

[1] K. R. K. M., K. K. Poojary, Abhay, A. R. S., and L. K. S. R., "AI-driven DevOps for intelligent automation in continuous software delivery pipelines," in *Proc. 5th Int. Conf. Evol. Comput. Mobile Sustain. Netw. (ICECMSN)*, 2025, pp. 433–440, doi: 10.1109/ICECMSN68058.2025.11382867.

[2] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston, MA, USA: Addison-Wesley, 2010.

[3] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux J.*, vol. 2014, no. 239, 2014.

[4] Prometheus Authors, "Overview," Prometheus Documentation. Accessed: Sep. 30, 2026. [Online]. Available: https://prometheus.io/docs/introduction/overview/

[5] The Kubernetes Authors, "Horizontal pod autoscaling," Kubernetes Documentation. Accessed: Sep. 30, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

[6] Grafana Labs, "About Grafana," Grafana Documentation. Accessed: Sep. 30, 2026. [Online]. Available: https://grafana.com/docs/grafana/latest/introduction/

[7] Docker Inc., "What is Docker?" Docker Documentation. Accessed: Sep. 30, 2026. [Online]. Available: https://docs.docker.com/get-started/docker-overview/

[8] Git SCM, "Git reference." Accessed: Sep. 30, 2026. [Online]. Available: https://git-scm.com/docs

[9] GitHub, "Understanding GitHub Actions," GitHub Docs. Accessed: Sep. 30, 2026. [Online]. Available: https://docs.github.com/actions

[10] GitLab, "CI/CD YAML syntax reference," GitLab Docs. Accessed: Sep. 30, 2026. [Online]. Available: https://docs.gitlab.com/ci/yaml/

[11] Jenkins, "Pipeline," Jenkins User Documentation. Accessed: Sep. 30, 2026. [Online]. Available: https://www.jenkins.io/doc/book/pipeline/

[12] The Kubernetes Authors, "Deployments," Kubernetes Documentation. Accessed: Sep. 30, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

[13] The Kubernetes Authors, "Service," Kubernetes Documentation. Accessed: Sep. 30, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/services-networking/service/

[14] OpenTelemetry Authors, "OpenTelemetry specification." Accessed: Sep. 30, 2026. [Online]. Available: https://opentelemetry.io/docs/specs/otel/

[15] M. Souppaya, J. Morello, and K. Scarfone, *Application Container Security Guide*, NIST SP 800-190, Gaithersburg, MD, USA, 2017.

[16] OpenSSF, "Supply-chain levels for software artifacts (SLSA) specification, v1.0." Accessed: Sep. 30, 2026. [Online]. Available: https://slsa.dev/spec/v1.0/

[17] OWASP Foundation, "Software supply chain security cheat sheet," OWASP Cheat Sheet Series. Accessed: Sep. 30, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Software_Supply_Chain_Security_Cheat_Sheet.html

[18] National Telecommunications and Information Administration, *The Minimum Elements for a Software Bill of Materials (SBOM)*, U.S. Dept. of Commerce, Washington, DC, USA, 2021.

[19] M. Souppaya, K. Scarfone, and D. Dodson, *Secure Software Development Framework (SSDF) Version 1.1*, NIST SP 800-218, Gaithersburg, MD, USA, 2022.

[20] C. Pahl, "Containerization and the PaaS cloud," *IEEE Cloud Comput.*, vol. 2, no. 3, pp. 24–31, May/Jun. 2015.

[21] A. A. U. Rahman and L. Williams, "Software security in DevOps: Synthesizing practitioners' perceptions and practices," in *Proc. Int. Workshop Continuous Softw. Evol. Delivery (CSED)*, 2016, pp. 70–76, doi: 10.1145/2896941.2896946.

[22] Electron, "Introduction to Electron," Electron Documentation. Accessed: Sep. 30, 2026. [Online]. Available: https://www.electronjs.org/docs/latest/

[23] OpenJS Foundation, "Node.js documentation." Accessed: Sep. 30, 2026. [Online]. Available: https://nodejs.org/docs/latest/api/

[24] G. Novakova Nedeltcheva, A. De La Fuente Ruiz, L. Orue-Echevarria Arrieta, N. Bat, and L. Blasi, "Towards supporting the generation of infrastructure as code through modelling approaches: Systematic literature review," in *Proc. IEEE 19th Int. Conf. Softw. Archit. Companion (ICSA-C)*, 2022, pp. 210–217, doi: 10.1109/ICSA-C54293.2022.00048.

[25] Cloud Native Computing Foundation, "Cloud native security whitepaper." Accessed: Sep. 30, 2026. [Online]. Available: https://github.com/cncf/tag-security/tree/main/security-whitepaper